Goflix

A streaming platform: the admin dashboard and its system design

Enrico Saputra | Backend developer (Laravel dashboard)

Client code and data are private. The demo rebuilds the Laravel dashboard on the real data model, with made-up data.

What the product does

Viewers watch movies and series on web and mobile. Some titles are free; the rest are premium through a monthly or yearly plan, or rented one movie at a time.

My part: the admin dashboard

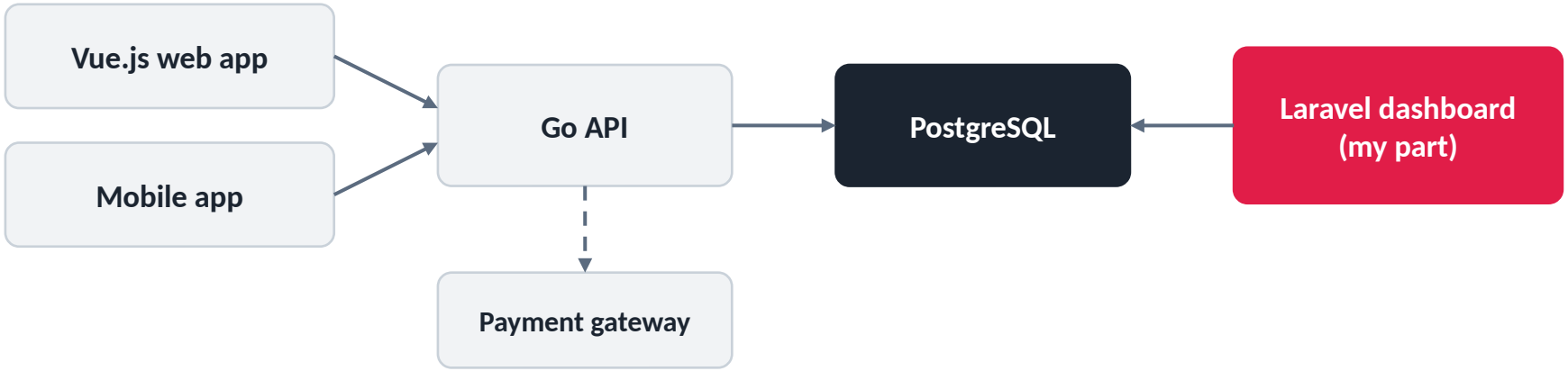
- Catalog: movies, series, episodes, genres, cast
- Subscription plans, coupons and rentals
- Users, payments and reporting

3 parts: Vue, Go, Laravel

1 shared PostgreSQL database

3 ways to watch: free, plan, rental

System overview



One database, two writers

The Go API writes activity and in-app payments; the dashboard writes the catalog, plans and corrections.

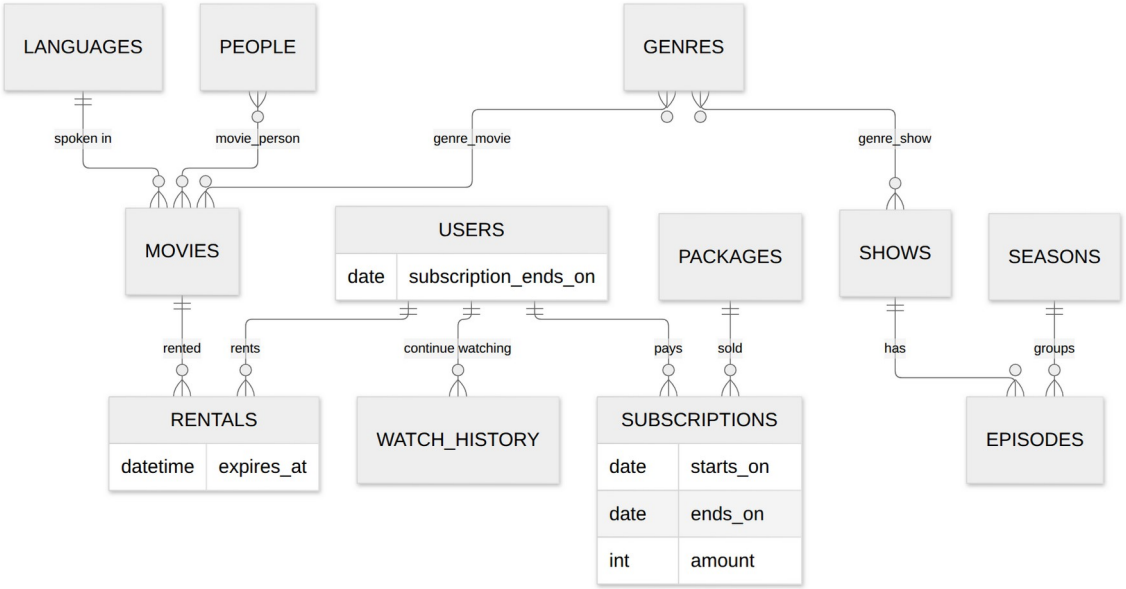
Server decides access

The app never decides on its own; the API checks. The dashboard shows the same rule to support.

The dashboard has no API

Everything the apps need goes through the Go API. The dashboard is admin pages only.

Data model



Catalog (movies, series, episodes) on the sides; users in the middle with payments and viewing history. users.subscription_ends_on is the source of truth for a fast access check.

Relations: from text to pivot tables

Production

```
video
  id: 12
  category_id: '6,12,19'
  cast_id: '1,108,109,110'
```

Cannot be joined or indexed; deleted ids stay behind.

Demo

```
genre_movie
  (movie_id 12, genre_id 6)
  (movie_id 12, genre_id 12)
  (movie_id 12, genre_id 19)
```

Foreign keys, normal joins, and the genre report is one query.

Gradual migration: add the pivot, fill it from the old column, move readers one by one, then drop the old column.

Renewing without losing paid days

Active until 20 Mar, buys 12 months on 10 Mar:

old: until 20 Mar	new: 21 Mar to 20 Mar next year

Ended 31 Jan, buys 1 month on 10 Mar:

lapsed	10 Mar to 9 Apr

One place

Only SubscriptionService moves users.subscription_ends_on.

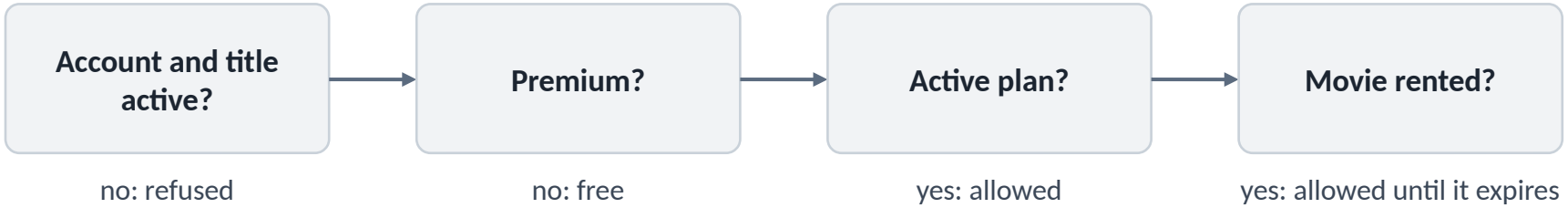
One transaction

The payment row and the end date are saved together, or neither is.

Row lock

lockForUpdate makes a second payment wait and count from the new date.

Who may watch what



Refused: needs a plan, or a rental if the movie has a rent price

An episode is premium on its own or through its series. One AccessService is used everywhere, and the dashboard shows it per user so support can answer complaints.

Rentals and coupons

Renting one movie

- Only premium movies with a rent price
- Lasts rent_days days from payment
- Active subscribers cannot rent: it is already included
- The same movie cannot be rented twice at once

Coupons

- Fixed amount or percentage
- Only valid between its start and end dates
- Never takes a price below zero
- A single-use coupon works once per user, across plans and rentals

Decisions worth talking about

End date on users

Deliberate duplication: the API checks access on every play, so one indexed column beats finding the latest subscription.

Payments cannot be edited

Subscriptions and rentals are financial records. A correction is a new entry.

Deactivate, not delete

A genre, plan, coupon or user in use can only be deactivated, so history stays intact.

Episodes updated in place

Saving the episode list keeps the same rows, so viewing history stays attached.

Tech stack and why

1. **Laravel 10** — Admin dashboard: forms, validation and CRUD are very productive. The part I built.
2. **PostgreSQL** — One database for the API and the dashboard, with transactions and row locks.
3. **Go** — API for the apps: high playback traffic and low latency.
4. **Vue.js** — The viewer web app.

Keeping the code easy to read

```
return DB::transaction(function () use (...) {
    $user = User::lockForUpdate()
        ->findOrFail($user->id);

    $startsOn = $user->subscription_ends_on
        ?->gte($today)
        ? $user->subscription_ends_on->addDay()
        : $today;
    $endsOn = $startsOn->copy()
        ->addMonthsNoOverflow($months)->subDay();

    $user->subscriptions()->create([...]);
    $user->update(['subscription_ends_on' => $endsOn]);
});
```

Rules in services

Thin controllers; plans, rentals, coupons and access are plain classes.

Enums, not numbers

Sign-in, role and coupon type are typed.

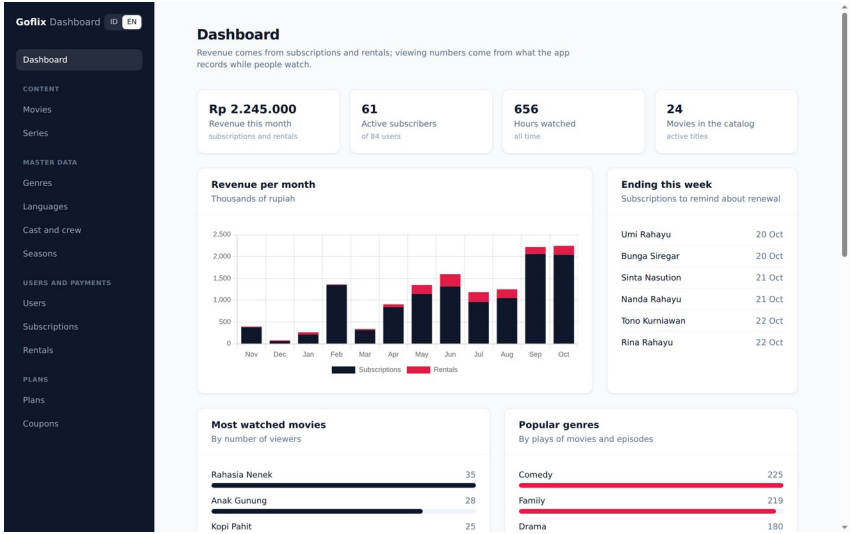
One pattern for master data

Genres, languages and seasons share one base controller.

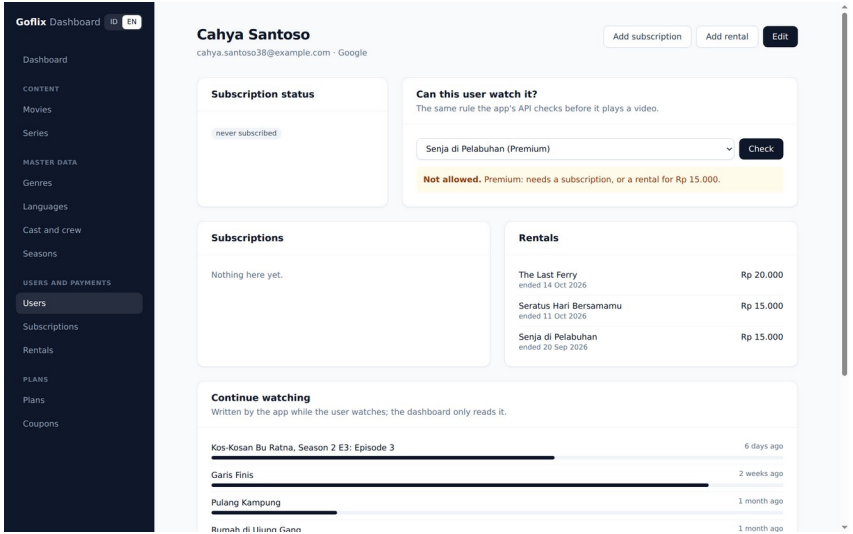
Tests on the rules

33 feature tests.

The demo app



Dashboard: plan and rental revenue per month



User page: status, payments and access check

What I would improve

Move to pivot tables

Replace the comma-separated ids step by step, with no downtime.

One shared rule contract

The access rule lives in Go and Laravel. One shared set of test cases keeps them in step.

Large viewing history

Partition watch_history by month and move reports to a daily summary table.

Constraints and indexes

Foreign keys and indexes on the columns used by access checks and reports.

Try it yourself

Live demo	https://goflix-dashboard-demo.vercel.app
Slides (PDF)	https://goflix-dashboard-demo.vercel.app/goflix-walkthrough.pdf
Slides (PPTX)	https://goflix-dashboard-demo.vercel.app/goflix-walkthrough.pptx
Demo code	Private GitHub repo, access given during the interview

The dashboard is available in Bahasa Indonesia and English. Demo data can be edited and is restored every 3 hours.